

JAVA 17 OCP · YDS STUDY SERIES

Unit 08 · Technical Memory Notes

· Lambdas and Functional Interfaces

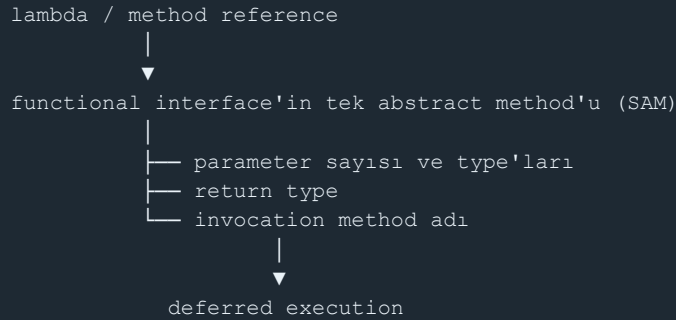
Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya [eksiksiz çift dilli Chapter 8 notundaki](#) kuralları active recall, karar tabloları ve kısa Java 17 örnekleriyle pekiştirir. Ana model şudur:

Lambda tek başına bir type değildir; **target type** olan functional interface, parameter ve return kurallarını lambda'ya verir.

Dil çalışması: [vocabulary](#) · [grammar notes](#).

1. Master map



Bir lambda declaration anında çalışmaz. Functional interface'in invocation method'u (`test()`, `apply()`, `accept()`, `get()` gibi) çağrıldığında body çalışır.

```

Predicate<String> empty = s -> s.isEmpty(); // logic şimdi tanımlandı
boolean result = empty.test("");             // logic burada çalıştı
  
```

2. Lambda syntax karar tablosu

Sol taraf: parameter list

Durum	Geçerli kısa biçim	Geçerli açık biçim
0 parameter	<code>() -> ...</code>	Başka biçim yok
1 inferred parameter	<code>x -> ...</code>	<code>(x) -> ...</code>
1 explicit type	—	<code>(String x) -> ...</code>
1 <code>var</code> parameter	—	<code>(var x) -> ...</code>
2+ parameter	—	<code>(x, y) -> ...</code>

Tek parameter'da parentheses yalnız **type implicit ise** atlanabilir.

```

Predicate<String> a = x -> x.isEmpty();
Predicate<String> b = (String x) -> x.isEmpty();
Predicate<String> c = (var x) -> x.isEmpty();
  
```

Parameter formatı bütünüyle aynı olmalı

```
(x, y) -> x.equals(y)           // geçerli: hepsi inferred
(String x, String y) -> x.equals(y) // geçerli: hepsi explicit
(var x, var y) -> x.equals(y)    // geçerli: hepsi var

(var x, y) -> true               // DOES NOT COMPILE
(String x, var y) -> true        // DOES NOT COMPILE
(String x, y) -> true            // DOES NOT COMPILE
```

Annotation veya `final`, explicit type/`var` biçiminde kullanılabilir:

```
(final var x, @Deprecated var y) -> x.compareTo(y)
```

Sağ taraf: expression body ve block body

Body	Braces	<code>return</code>	Son semicolon
Expression	Hayır	Hayır	Hayır
Void block	Evet	Genellikle hayır	Her statement'ta
Value-returning block	Evet	Evet	<code>return value;</code> içinde

```
x -> x.length()           // expression body
x -> { return x.length(); } // value-returning block
x -> { System.out.print(x); } // void block
x -> {}                   // geçerli empty void block
```

Şunlar farklı nedenlerle derlenmez:

```
x -> { x.length(); } // value bekleniyorsa return eksik
x -> { return x.length() } // return statement semicolon'u eksik
x -> return x.length(); // return, braces olmadan yazılamaz
```

OCP tuzağı: Expression statement olan bir çağrının döndürdüğü değer, `Consumer` gibi `void` hedefte discard edilebilir:

```
Map<String, Integer> map = new HashMap<>();
BiConsumer<String, Integer> put = map::put;
```

`Map.put()` değer döndürmesine rağmen invocation, void-compatible statement expression olarak kullanılabilir.

3. `var` ile iki farklı kural

Lambda parameter'ında `var` kullanılabilir:

```
Predicate<String> p = (var s) -> s.isEmpty();
```

Fakat lambda'nın kendisi doğrudan `var` variable'a atanamaz:

```
var p = (String s) -> s.isEmpty(); // DOES NOT COMPILE
```

Neden: Lambda da `var` da type'ını context'ten bekler; ortada target functional interface yoktur.

Hafıza cümlesi: `var`, lambda'nın içindeki parameter'a yardım eder; lambda'nın kendi type'ını veremez.

4. Functional interface ve SAM hesabı

Functional interface tam olarak bir abstract method contract'ına sahiptir. Hesap yaparken declaration sayısını değil, inherited method'lar birleştirildikten sonra kalan abstract contract sayısını düşün.

Sayıma katılanlar

- Interface'te bildirilen abstract method
- Parent interface'ten inherit edilen ve başka method'la override-equivalent olmayan abstract method

Sayıma katılmayanlar

- `default` method
- `static` method
- `private` method
- `private static` method
- `Object`te public olarak bulunan signature'larla eşleşen `equals(Object)`, `hashCode()` ve `toString()` declaration'ları

```
@FunctionalInterface
interface Dive {
    String toString();           // Object signature: sayılmaz
    boolean equals(Object o);    // Object signature: sayılmaz
    int hashCode();              // Object signature: sayılmaz
    void dive();                 // tek SAM
}
```

`equals(MyType)` farklı signature'dır ve sayılır.

```
interface NotSam {
    boolean equals(NotSam other); // abstract #1
    void run();                   // abstract #2
}
```

@FunctionalInterface

- Annotation optional'dır.
- Varsa compiler SAM kuralını enforce eder.
- Annotation yokluğu functional interface olmayı engellemez.
- Class functional interface değildir; functional interface bir interface type'tır.
- Java 17'de `sealed` olarak bildirilen bir interface, tek abstract method'u olsa bile functional interface değildir ve lambda target type'ı olamaz.

```
@FunctionalInterface
interface Dance {
    void move();
    void rest();
} // DOES NOT COMPILE: iki abstract method
```

5. Method reference: dört biçim, tek çeviri yöntemi

Tür	Method reference	Long-form lambda
Static method	<code>Math::round</code>	<code>x -> Math.round(x)</code>
Particular object instance method	<code>str::startsWith</code>	<code>x -> str.startsWith(x)</code>
Parameter üzerindeki instance method	<code>String::isEmpty</code>	<code>s -> s.isEmpty()</code>
Constructor	<code>String::new</code>	<code>x -> new String(x)</code>

Unbound instance method'da ilk parameter receiver'dır

```
BiPredicate<String, String> starts = String::startsWith;
// (text, prefix) -> text.startsWith(prefix)
```

İlk SAM parameter'ı method'un çağrılacağı object; geri kalanlar gerçek method argument'larıdır.

Overload'u reference değil target type çözer

```
interface Converter {
    long round(double value);
}
Converter c = Math::round;
```

`Math.round(float)` ve `Math.round(double)` overloaded olsa da `Converter` signature'ı `double -> long` sürümünü seçer. Uygun hiç overload yoksa veya birden fazla overload eşit derecede uygunsa kod derlenmez.

Her lambda method reference olamaz

Method reference syntax'ı normal bir method argument'ını önceden sabitleyemez. Particular-object biçiminde receiver'ın bağlanması bunun ayrı bir durumudur:

```
String value = "";
Supplier<Boolean> lambda = () -> value.startsWith("Zoo");
```

Buradaki "Zoo" argument'ını `value::startsWith` içine gömecek bir syntax yoktur.

Evaluation caveat: `value::startsWith` oluşturulurken `value` receiver expression'ı hemen değerlendirilir; yalnız method invocation ertelenir. Receiver o anda `null` ise functional method sonradan çağrılmadan önce method reference oluşturulurken `NullPointerException` oluşur.

Analiz sırası:

- 1 Target SAM signature'ını yaz.
- 2 `::` solunun class mı object mi olduğunu belirle.
- 3 Unbound instance method ise ilk SAM parameter'ını receiver olarak ayır.
- 4 Kalan parameter'ları method/constructor argument'larına sırayla geçir.
- 5 Return compatibility'yi kontrol et.

6. Built-in functional interface ana matrisi

Amaç	Akış	Interface	SAM
Üret	<code>() -> T</code>	<code>Supplier<T></code>	<code>T get()</code>
Tüket	<code>T -> void</code>	<code>Consumer<T></code>	<code>void accept(T)</code>
İki değeri tüket	<code>T,U -> void</code>	<code>BiConsumer<T,U></code>	<code>void accept(T,U)</code>
Koşul test et	<code>T -> boolean</code>	<code>Predicate<T></code>	<code>boolean test(T)</code>
İki değer test et	<code>T,U -> boolean</code>	<code>BiPredicate<T,U></code>	<code>boolean test(T,U)</code>
Dönüştür	<code>T -> R</code>	<code>Function<T,R></code>	<code>R apply(T)</code>
İki input'u dönüştür	<code>T,U -> R</code>	<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>
Aynı type'ı dönüştür	<code>T -> T</code>	<code>UnaryOperator<T></code>	<code>T apply(T)</code>
Aynı type'taki iki değeri birleştir	<code>T,T -> T</code>	<code>BinaryOperator<T></code>	<code>T apply(T,T)</code>

Dört çekirdek hafıza fiili

```
Supplier → get
Consumer → accept
Predicate → test
Function → apply
```

Var olan `Bi...` biçimlerinde `Bi`, çekirdek interface'e bir input daha ekler. Ancak bu, her interface'in `Bi` sürümü olduğu anlamına gelmez: `BiSupplier` ve `BiUnaryOperator` yoktur. `UnaryOperator`, `Function<T,T>` nin; `BinaryOperator` ise `BiFunction<T,T,T>` nin more specific hâlidir.

boolean primitive ile Boolean object'i ayır

```
Predicate<String> p;           // String -> boolean
Function<String, Boolean> f;    // String -> Boolean
```

Soru **Boolean** object diyorsa yalnızca “true/false dönüyor” diye hemen **Predicate** seçme.

7. Primitive specialization şablonları

Primitive specialization'lar boxing maliyetini azaltır ve adları type akışını gösterir.

Aynı primitive ailesi

```
IntSupplier      ()      -> int      getAsInt ()
IntConsumer      int     -> void     accept ()
IntPredicate      int     -> boolean  test ()
IntFunction<R>    int     -> R        apply ()
IntUnaryOperator  int     -> int      applyAsInt ()
IntBinaryOperator int,int -> int      applyAsInt ()
```

Double... ve **Long...** aileleri aynı pattern'ı izler.

Adı soldan sağa oku

Interface	Akış
ToIntFunction<T>	T -> int
ToIntBiFunction<T,U>	T,U -> int
DoubleToIntFunction	double -> int
IntToLongFunction	int -> long
ObjIntConsumer<T>	T,int -> void

Adında **Boolean** bulunan tek primitive specialization **BooleanSupplier**dir:

```
BooleanSupplier condition = () -> true;
boolean result = condition.getAsBoolean();
```

CharSupplier, **FloatSupplier**, **IntegerSupplier** gibi isimleri Java API'sinde varmış gibi kabul etme.

8. Convenience methods ve evaluation order

Predicate

```
Predicate<String> egg = s -> s.contains("egg");
Predicate<String> brown = s -> s.contains("brown");

Predicate<String> brownEgg = egg.and(brown);
Predicate<String> anyEgg = egg.or(brown);
Predicate<String> notBrown = brown.negate();
```

`and()` ve `or()` short-circuit evaluation uygular.

Consumer

```
Consumer<String> first = s -> System.out.print("A");
Consumer<String> second = s -> System.out.print("B");
first.andThen(second).accept("x"); // AB
```

Aynı input önce `first`, sonra `second` için kullanılır.

Function

```
Function<Integer,Integer> plusOne = x -> x + 1;
Function<Integer,Integer> timesTwo = x -> x * 2;
```

Composition	<code>apply()</code> akışı	Sonuç
<code>timesTwo.compose(plusOne)</code>	$(3 + 1) * 2$	8
<code>plusOne.andThen(timesTwo)</code>	$(3 + 1) * 2$	8
<code>plusOne.compose(timesTwo)</code>	$(3 * 2) + 1$	7

Hafıza cümlesi: `compose(before)` önce argument'taki function'ı; `andThen(after)` sonra argument'taki function'ı çalıştırır.

9. Variable scope ve capture

Variable	Lambda body'sinden kullanım
Instance variable	İzin verilir; mutable olabilir
Static variable	İzin verilir; mutable olabilir
Local variable	Yalnız <code>final</code> veya effectively final
Method parameter	Yalnız <code>final</code> veya effectively final
Lambda parameter	İzin verilir

Effectively final testi

Initializer ile değer alan variable, initialization'dan sonra yeniden assign edilmiyorsa effectively finaldır. Initializer olmadan bildirilen local variable da her execution path'ta tam bir kez değer alıp sonrasında değişmiyorsa effectively final olabilir.

```
void call(String name) {
    Consumer<String> c = x -> System.out.println(name); // geçerli
}
```

```
void call(String name) {
    name = "new";
    Consumer<String> c = x -> System.out.println(name); // DOES NOT COMPILE
}
```

Reassignment lambda'dan sonra yazılsa bile bütün method scope'u analiz edilir:

```
int volume = 1;
Supplier<Integer> s = () -> volume; // DOES NOT COMPILE
volume = 2;
```

Compiler error genellikle assignment satırında değil, capture'ın yapıldığı lambda satırında raporlanır.

Shadowing

Lambda parameter, çevreleyen local variable veya method parameter ile aynı adı alamaz:

```
void test(int age) {
    Predicate<Integer> p = age -> age > 0; // DOES NOT COMPILE
}
```

Lambda body içindeki local variable da lambda parameter'ını redeclare edemez:

```
(a, b) -> { int a = 0; return b; } // DOES NOT COMPILE
```

Fakat birbirinden ayrı iki lambda scope'unda aynı parameter adı kullanılabilir; type'ları target interface'e göre farklı bile olabilir.

10. Compile-time, runtime ve output ayrımı

Durum	Sonuç
Target type yok: <code>var x = a -> a;</code>	Does not compile
SAM sayısı iki	Lambda target olamaz; annotation varsa interface declaration da derlenmez
Lambda parameter style karışık	Does not compile
Captured local yeniden atanmış	Lambda kullanım satırında does not compile
Uygun SAM ve body	Derlenir; invocation method çağrılınca çalışır
Bound method reference'ın receiver'ı <code>null</code>	Derlenir; reference oluşturulurken <code>NullPointerException</code>
<code>LocalDate.now()</code> kullanan supplier	Derlenir; sonuç çalıştırıldığı tarihe göre değişir
<code>Math.random()</code> kullanan lambda	Derlenir; output çalıştırmaya göre değişir
Lambda object'inin <code>toString()</code> i	Derlenir; generated görünüm portable değildir
<code>HashMap</code> yazdırma	Entry order garanti edilmez

Kaynak terminolojisi ve örnek output caveat'leri

- Kaynağın PDF sayfa 428'de "Soar class" dediği declaration aslında bir `interface` tir.
- PDF sayfa 440'taki `Function–BiFunction` ilişkisi için doğru terim **superinterface**'tir; interface'lerin superclass'ı olmaz.
- PDF sayfa 444'te `BooleanSupplier` örneğinin yanındaki `false`, garanti edilen output değil örnek bir çalıştırma sonucudur; `Math.random()` nedeniyle `true` da yazdırılabilir.
- `LocalDate.now()` sonucu tarihe, lambda object'inin `toString()` görünümü JVM implementation'ına, `HashMap` entry sırası ise garanti edilmeyen iteration order'a bağlıdır.

11. OCP için 25 saniyelik çözüm algoritması

- 1 Kodun lambda için target functional interface verip vermediğini bul.
- 2 Interface'teki gerçek SAM sayısını hesapla.
- 3 SAM parameter sayısı/type'ları ile lambda'nın solunu karşılaştır.
- 4 Parentheses, explicit type ve `var` formatını kontrol et.
- 5 Body expression mı block mu; `return`, braces ve semicolon uyumlu mu?
- 6 Return type primitive mi wrapper mı?
- 7 Method reference ise long-form lambda'ya çevir.
- 8 Captured local/method parameter `final` veya effectively final mı?
- 9 Aynı scope'ta variable adı tekrar kullanılmış mı?
- 10 En son compile-time / runtime / output sonucunu ayrı ayrı yaz.

12. Aktif hatırlama · Özgün OCP tarzı çalışma soruları

Bu sorular çalışma amacıyla yazılmıştır; gerçek sınav sorusu değildir.

- 1 `Predicate<String> p = (var s) -> s.isBlank();` Java 17'de derlenir mi?
- 2 `var p = s -> s.isEmpty();` neden derlenmez?
- 3 `String::startsWith, BiPredicate<String, String>` target'ında hangi long-form lambda'ya dönüşür?
- 4 `Function<Integer, Integer> a = x -> x + 2;` ve `b = x -> x * 3;` için `a.compose(b).apply(2)` kaçtır?
- 5 `int n = 1; Supplier<Integer> s = () -> n; n++;` hangi aşamada hata verir?
- 6 `interface X { String toString(); void run(); }` functional interface midir?
- 7 `IntFunction<String>` input ve output type'ları nelerdir?
- 8 `(var a, b) -> a.equals(b)` neden geçersizdir?

Cevaplar ve açıklamalar

- 1 Evet. Target `Predicate<String>` type'ı verir; tek `var` parameter parentheses içindedir ve body `boolean` döndürür.
- 2 Hem `var` hem lambda type bilgisini context'ten bekler; target functional interface yoktur.
- 3 `(text, prefix) -> text.startsWith(prefix).`
- 4 8. `compose(b)` önce `byi` çalıştırır: `2 * 3 = 6`; sonra `a: 6 + 2 = 8`.
- 5 Kod derlenmez. `n++`, `n`yi effectively final olmaktan çıkarır; compiler lambda'nın `n`yi capture ettiği yerde hata raporlar.
- 6 Evet. `toString()` public `Object` signature'ıyla eşleştiği için SAM sayımına katılmaz; tek abstract contract `run()` dir.
- 7 Input `int`, output `String`; SAM `String apply(int value)` biçimindedir.
- 8 Parameter formatları karışıktır. Ya `(a, b)` ya da `(var a, var b)` kullanılmalıdır.

Son tekrar kartı

```

Target type → SAM → parameter → body → return → capture

O→T      Supplier.get
T→void    Consumer.accept
T→boolean Predicate.test
T→R       Function.apply

:: solunda object → particular object
:: solunda class  → static veya unbound instance; SAM çözer

local/method capture → final/effectively final
instance/static       → mutable olsa da kullanılabilir

```